

Load Balancing and Expert Specialisation in Mixture-of-Experts Routing: A Pilot Study

Muad Mazahir
muadmazahir@gmail.com

Draft. Work in progress.

Abstract

Standard frontier Mixture-of-Experts (MoE) models are generally much larger in terms of total parameter count than models of similar capability. Their advantage comes from the fact that during any given forward pass only a subset of parameters are activated, thereby making the model more efficient from a compute standpoint. The major disadvantage is that because the overall model is much larger, it requires substantially more memory, as the whole model has to be loaded into memory during inference. This high memory demand comes with all sorts of challenges. We contend that this large memory demand is due to the way these models are trained and that the models can be made smaller without sacrificing the gains on compute. During training, MoE models face a problem referred to as router collapse where the model continually improves one or a few experts while leaving the others largely untrained. To prevent router collapse, the go to solution is to use an auxiliary load-balancing loss which penalises imbalanced routing and forces all experts to get trained. But a router that routes at random is just as balanced as one that has learned a meaningful partition of the input, and the loss cannot tell these two ways of achieving load balance apart. We argue that routers that are both balanced and specialised should, in principle, allow each expert to be made much smaller. We also posit five alternative training techniques for preventing router collapse that do not make use of an auxiliary loss, namely, epsilon-greedy exploration outside the top- k , gradient accumulation, a straight-through estimate of unevaluated experts, a confidence-gated dense fallback, and threshold-based routing. We also do some evaluations comparing them against an auxiliary-loss baseline and an unmitigated control in a small, carefully budget-matched character-level language-modelling testbed. For our current setup and scale, auxiliary loss seems like the best approach based on quality, balance and token dropping; each alternative helps in some regimes and hurts in others; and the collapse metrics we report, like those typically used in the literature, depend only on how load is spread across experts, so none of them can distinguish specialised from random routing. Because our compute did not allow a conclusive test of the specialisation hypothesis, we contribute an evaluation protocol, including specialisation metrics and an expert-shrinking experiment. Code is available at <https://github.com/muadmazahir/MoE-experiments>.

1 Introduction

Mixture-of-Experts (MoE) layers [14, 35] let a model hold far more parameters than it uses for any one token. A learned router sends each token to k of E feed-forward “experts”, so the model only pays for k of them. Many of today’s strongest open language models are built this way: DeepSeek-V3 activates 37B of its 671B parameters per token [4], and Kimi K2 activates 32B of roughly one trillion [18].

The catch is memory. Any token may be routed to any expert, so every expert has to be held in memory at inference, and a model that uses 37B parameters per token still needs memory for all 671B. That trained experts can be pruned or merged with only a small loss in quality [21, 23] suggests that trained MoEs contain a lot of redundancy.

This paper asks whether that redundancy comes from how MoEs are trained. Left unchecked, top- k routing *collapses*: the router sends more tokens to the experts it already slightly favours, those experts get more training and improve, and the router favours them even more, while the experts it skips get no gradient from the tokens they miss [8, 35]. The standard fix is an auxiliary load-balancing loss [8, 20] that penalises the router when some experts get more tokens than others. It works, but it only rewards an *even spread* of tokens, and a router that assigns tokens at random is just as evenly spread as one that has learned a meaningful partition of the input.

If routing is close to random, every expert sees roughly the same mix of tokens and learns roughly the same function, so each has to be a full-size generalist. If instead each expert handled its own coherent slice of the inputs, it would only

need enough capacity for that slice, and the experts could in principle be made much smaller. That is the hypothesis behind this work: *preventing collapse and producing specialisation are different goals, and the second is what would let MoE models shrink.*

Contributions.

1. We describe six strategies for preventing collapse, five of which use no auxiliary loss, with the mathematics of each and an analysis of how each can fail (section 4).
2. We compare them in a small, budget-matched testbed across three regimes (sections 5 and 6). The auxiliary loss is the strongest single method at this scale; the alternatives each carry a useful mechanism and a failure mode, and several failure modes are only visible per layer.
3. We lay out the experimental protocol that would actually test the specialisation-and-shrinkage hypothesis, including specialisation metrics and an iso-memory expert-shrinking experiment, together with a compute estimate (section 8).

All experiments are single-seed runs of a 2.4M-parameter model on a 1M-character corpus. They are enough to expose mechanisms and failure modes, but not to rank methods whose quality differs by a few percent, or to test the hypothesis itself. We also do not measure specialisation: every metric we report describes only how evenly load is spread across experts, and measuring specialisation is left to the protocol in section 8. We present the experiments as a pilot study.

2 Related work

Sparse MoE layers were introduced by Shazeer et al. [35] and scaled up by GShard [20] and the Switch Transformer [8], whose auxiliary loss we study. Several recent papers suggest that balancing and specialisation pull against each other. Qiu et al. [31] show that computing the balancing loss per micro-batch forces near-uniform routing, and that balancing over the global batch instead yields more specialised experts and better models. Omi et al. [29] and Guo et al. [10] link standard balancing to overly uniform routing and redundant experts, and Falke et al. [6] find in a small-scale testbed that the scope of balancing decides whether specialisation can coexist with full use of the experts. DeepSeek’s auxiliary-loss-free strategy [4, 38] avoids the balancing loss altogether by adding a per-expert bias to the routing scores. At the same time, hash and random routing are competitive with learned routing [7, 34, 43], which suggests that current routers learn little beyond balance. The closest prior work to each strategy we test is cited alongside it in section 4.

3 Problem formulation

3.1 The MoE layer

A token arrives at the layer as $x \in \mathbb{R}^d$. A linear router scores every expert and a softmax turns the scores into a distribution:

$$z = W_r x \in \mathbb{R}^E, \quad p = \text{softmax}(z), \quad p_i = \frac{e^{z_i}}{\sum_{j=1}^E e^{z_j}}. \quad (1)$$

A routing rule selects a set $S(x) \subseteq \{1, \dots, E\}$, normally the top k . The layer’s output is the sum of the selected experts’ outputs, each scaled by a *gate* g_i computed from the router probabilities:

$$y = \sum_{i \in S(x)} g_i \text{FFN}_i(x), \quad \text{FFN}_i(x) = W_2^{(i)} \text{GELU}(W_1^{(i)} x). \quad (2)$$

The gate is either *renormalised*, so that the selected experts’ weights sum to one (GShard, Mixtral), or *raw*, using the router probability directly (Switch):

$$g_i^{\text{renorm}} = \frac{p_i}{\sum_{j \in S} p_j}, \quad g_i^{\text{raw}} = p_i, \quad (3)$$

so the two differ only in whether the selected probabilities are rescaled. Over a batch of N tokens, let $\mathcal{A}$ be the multiset of (token, expert) assignments made ($|\mathcal{A}| = Nk$ for top- k). The *dispatch load* and *router mass* are

$$f_i = \frac{1}{|\mathcal{A}|} |\{(t, j) \in \mathcal{A} : j = i\}|, \quad \bar{p}_i = \frac{1}{N} \sum_{t=1}^N p_{t,i}. \quad (4)$$

An optional capacity factor γ limits each expert to $C = \lfloor \gamma |\mathcal{A}| / E \rfloor$ assignments per batch; tokens that overflow skip the expert and pass through the residual connection only [8, 20]. Unlike those works, which drop overflow in token order, we drop the lowest-gate assignments first, in the spirit of the batch prioritized routing of Riquelme et al. [33].

3.2 Collapse

For $j \notin \mathcal{S}(x)$ under a renormalised gate, both the expert and its router logit receive no gradient from token x :

$$\frac{\partial \mathcal{L}}{\partial W^{(j)}} = 0, \quad \frac{\partial \mathcal{L}}{\partial z_j} = 0. \quad (5)$$

Nothing, therefore, corrects an early accidental preference; the loop amplifies it. We measure collapse from the dispatch histogram on held-out data: the normalised load entropy $H_{\text{norm}} = H(f) / \ln E$, where 1 is perfectly balanced [25, 41]; the busiest expert’s share $\max_i f_i$, which Shazeer et al. [35] report as a ratio to the mean load and which is a monotone transform of the MaxVio metric of Wang et al. [38]; and the number of *dead* experts with $f_i < 0.1/E$. The dead-expert threshold is our choice; Wang et al. [40] use a similar cut-off of $1/64$ of tokens to mark inactive experts. Each metric is computed both pooled over layers and per layer.

3.3 Balance versus specialisation

Collapse metrics are functions of f (or $\bar{p}$) alone. They describe how *evenly* load is spread, not *whether the router is using its input*. The natural quantity for the latter is the mutual information between the token and the expert it is routed to, which ModuleFormer and LIMoE use as a training objective [27, 36]. Treating the router as a conditional distribution $p(i | x)$ and the token distribution as empirical over a batch,

$$I(X; \mathcal{E}) = \underbrace{H(\bar{p})}_{\text{balance}} - \underbrace{\frac{1}{N} \sum_{t=1}^N H(p_t)}_{\text{per-token indecision}} \in [0, \ln E]. \quad (6)$$

The first term is maximised by balance. The second is minimised by a router that is decisive about each token. $I(X; \mathcal{E}) = \ln E$ is achieved only by a router that is both perfectly balanced and deterministic, i.e. one that partitions the inputs into E equal cells; $I = 0$ is achieved by any router whose output ignores its input, including uniform-at-random routing. We call a router *specialised* to the extent that $I(X; \mathcal{E})$ is high, and additionally require (section 8) that the partition be stable and functionally meaningful, since I alone is also maximised by an arbitrary hash.

Definition 1 (Balance and specialisation). A router is *balanced* if $H(\bar{p}) \approx \ln E$ (equivalently $H_{\text{norm}} \approx 1$ for its dispatch) and *specialised* if, in addition, $\frac{1}{N} \sum_t H(p_t) \ll \ln E$.

The Switch auxiliary loss is

$$\mathcal{L}_{\text{aux}} = \alpha E \sum_{i=1}^E f_i \bar{p}_i \in [\alpha, \alpha E]. \quad (7)$$

This loss reaches its minimum, α , when both the load and the router mass are spread evenly, $f = \bar{p} = u$. That condition fixes the first term of (6) but says nothing about the second. A router that ignores its input and assigns tokens at random meets it, and so does one that splits the input cleanly and evenly between experts, so the loss scores the two the same even though their mutual information lies at opposite ends of its range.

At its minimum the loss is therefore indifferent to specialisation; the gradient shows how it acts away from the minimum. Treating f as constant (it is not differentiable),

$$\frac{\partial \mathcal{L}_{\text{aux}}}{\partial z_{t,j}} = \frac{\alpha E}{N} p_{t,j} \left(f_j - \sum_i p_{t,i} f_i \right). \quad (8)$$

The direction depends on the token only through p_t ; the *target* is the same global f for every token. The loss pushes each token away from experts that are popular *in this batch*, regardless of whether that token belongs there. When the batch is small or domain-homogeneous, a specialised router naturally produces a skewed f , and (8) penalises exactly that skew. This is the mechanism behind the finding of Qiu et al. [31] that micro-batch balancing suppresses domain specialisation. Nothing in (7) rewards reducing $H(p_t)$.

3.4 From specialisation to smaller experts

Each expert in our model has $2dd_h$ parameters, so a layer holds $2Edd_h$ expert parameters but activates only $2kdd_h$ per token. Memory scales with $E d_h$; compute with $k d_h$. Fine-grained designs [3, 11, 24] increase E and decrease d_h at fixed total parameters. The question we pose is different: can the *total* $E d_h$ be reduced at fixed quality?

Hypothesis 1 (Specialisation permits shrinkage). *Let $Q(d_h; \mathcal{R})$ be the quality of an MoE whose experts have width d_h , trained with routing procedure $\mathcal{R}$. For routers with low $I(X; \mathcal{E})$, experts receive statistically exchangeable token streams and converge toward redundant generalists, so Q degrades quickly as d_h shrinks. For routers with high and stable $I(X; \mathcal{E})$, each expert need only model a sub-distribution, so Q degrades slowly, and a substantially smaller $E d_h$ attains the same quality.*

The intuition is a mixture-of-regressions argument: if the target function restricted to cell X_i has lower complexity than on all of X , an expert that only sees X_i needs fewer parameters to fit it. Indirect support comes from compression results: that experts can be merged [21] or pruned [23] with limited loss implies that trained experts overlap substantially, which is what hypothesis 1 predicts for weakly specialised routing. We stress that hypothesis 1 is *untested* here; testing it is the purpose of the protocol in section 8.

4 Strategies for preventing collapse

We compare an unmitigated control and six strategies. Only one of them, *aux*, changes the training objective. Default hyper-parameters are listed in table 5.

4.1 Control (*none*)

Plain top- k , $\mathcal{S}(x) = \text{Top-}k(p)$, no additional loss. Without it one cannot tell a strategy that works from a configuration in which collapse never happened.

4.2 Auxiliary load-balancing loss (*aux*)

$\mathcal{L} = \mathcal{L}_{\text{task}} + \mathcal{L}_{\text{aux}}$ with $\mathcal{L}_{\text{aux}}$ from (7), $\alpha = 10^{-2}$ [8]. The gradient (8) pushes probability mass off overloaded experts, directly countering the feedback loop. The weakness of the loss itself (section 3.3) is that any router that spreads tokens evenly minimises it, including one that assigns them at random. The weight α sets the trade-off between balance and the task loss.

4.3 Epsilon-greedy exploration outside the top- k (*explore*)

Inspired by bandit exploration [37] and the noisy gating of Shazeer et al. [35], but with a structural twist: with probability ϵ a token abandons its top- k entirely and is instead sent to k different experts chosen at random from those outside its top- k ,

$$\mathcal{S}(x) = \begin{cases} \text{Top-}k(p) & \text{w.p. } 1 - \epsilon, \\ \text{uniform } k\text{-subset of } \{1, \dots, E\} \setminus \text{Top-}k(p) & \text{w.p. } \epsilon, \end{cases} \quad \epsilon(s) = \epsilon_0 + \min\left(1, \frac{s}{\rho S}\right)(\epsilon_1 - \epsilon_0). \quad (9)$$

Here s is the training step and S the total number of steps, so ϵ falls linearly from ϵ_0 to ϵ_1 over the first ρS steps and then stays at ϵ_1 . Choosing only from outside the top- k ensures every exploration step tests an expert the router had rejected. Exploration alone is ineffective, because a written-off expert has $p_j \approx 0$, so its gate is tiny and so is its router

gradient. We therefore soften the gate of exploring tokens with a *temperature* $T > 1$, dividing their logits by T before the softmax:

$$g_i^{\text{renorm}} = \frac{e^{z_i/T}}{\sum_{j \in S} e^{z_j/T}}, \quad g_i^{\text{raw}} = \frac{e^{z_i/T}}{\sum_{j=1}^E e^{z_j/T}}. \quad (10)$$

At $T = 1$ these are the gates of (3). For a selected expert i , the gradient that reaches its router logit through the gates is, for either gate,

$$\frac{\partial \mathcal{L}}{\partial z_i} = \frac{g_i}{T} \left\langle \frac{\partial \mathcal{L}}{\partial y}, \text{FFN}_i(x) - y \right\rangle, \quad (11)$$

which is proportional to the gate itself. A written-off expert with $g_i \approx 0$ therefore gets almost no signal, and raising T raises its gate. The temperature is applied only to exploring tokens. An exploring token replaces all k of its experts with random ones, so every expert in its gate uses the same temperature, and softened and unsoftened scores never appear in the same softmax. Every other token uses $T = 1$, exactly as at evaluation, so the trained model is unaffected by the temperature. With logits $[4, -16]$, the rejected expert’s gate rises from 2.1×10^{-9} at $T = 1$ to 7.6×10^{-2} at $T = 8$. A uniform gate *floor* would not work: it is constant in z , so it revives the expert but leaves the router frozen. Remaining concern: the signal still scales with $e^{z_j/T}$, so the most neglected experts are still the least informed about.

4.4 Gradient accumulation (accum)

Instead of updating the model after every batch, we run $M = 8$ batches, average their losses, and then make a single update:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \left(\frac{1}{M} \sum_{m=1}^M \mathcal{L}_m \right), \quad (12)$$

where θ are the model’s parameters, η is the learning rate and $\mathcal{L}_m$ is the loss on the m -th batch. Each update is therefore based on M times as many tokens. The idea is that this makes the router’s updates less noisy (their variance falls by roughly a factor of M), so a small, accidental preference for one expert in a single batch is less likely to snowball into collapse.

This is the only strategy that leaves the MoE layer itself unchanged, so how it compares depends on how the budget is fixed. With the same number of tokens, it makes M times fewer updates (375 instead of 3,000 in Exp. 1). With the same number of updates, it sees M times more tokens and uses about M times the compute (Exp. 3). We report both. It is also different from the global-batch balancing of Qiu et al. [31], which computes the balancing loss over a larger batch; here there is no balancing loss at all.

4.5 Straight-through estimate of unevaluated experts (surrogate)

Give the router gradient for all E experts at zero extra expert cost by estimating the outputs of the $E - k$ that did not run. Assume $\text{FFN}_i(x) \approx c p_i$ with one slope per token fitted by least squares on the evaluated experts:

$$c = \frac{\sum_{i \in S} p_i \text{FFN}_i(x)}{\sum_{i \in S} p_i^2}, \quad \sum_{j \notin S} p_j \text{FFN}_j(x) \approx c \sum_{j \notin S} p_j^2. \quad (13)$$

With $g_i \propto p_i$ the numerator of c equals κy , where $\kappa = 1$ for the raw gate and $\kappa = \sum_{i \in S} p_i$ for the renormalised gate, so the correction reduces to a per-token scalar $s = \kappa \sum_{j \notin S} p_j^2 / \sum_{i \in S} p_i^2$, wired in straight-through [1]:

$$\hat{y} = y + \alpha_s \text{sg}(y)(s - \text{sg}(s)). \quad (14)$$

The forward value is unchanged ($\hat{y} = y$); the backward pass sends $\partial \mathcal{L} / \partial s = \langle \partial \mathcal{L} / \partial y, \text{sg}(y) \rangle$ to every logit. Compared with Default MoE [30], which uses a per-expert running average, this estimate is collinear with y : it tells the router *how much* mass belongs off its top- k but not *which* unselected expert deserves it, and its gradient decays as p_j^2 . It also reaches only the router, never the starving expert’s weights.

4.6 Confidence-gated dense fallback (conf)

In ordinary top- k routing, collapse is driven by forced guesses: even when the router cannot yet tell the experts apart, top- k still commits the token to whichever experts are marginally ahead. In this strategy, we instead route a token densely, to every expert, whenever its top- k mass falls below a threshold. This threshold anneals to zero over training:

$$\mathcal{S}(x) = \begin{cases} \{1, \dots, E\} & \text{if } \sum_{i \in \text{Top-}k(p)} p_i < \tau_c(s), \\ \text{Top-}k(p) & \text{otherwise,} \end{cases} \quad \tau_c(s) = \tau_c^0 \left(1 - \min\left(1, \frac{s}{0.8S}\right)\right). \quad (15)$$

The useful range is $(k/E, 1]$, since top- k always holds at least k/E of the mass. It is a dense warm-up [28] in which tokens graduate individually. Its limitation is that being confident is not the same as being right. A collapsed router is a confident one, so it never triggers the fallback. The method can stop the router from committing too early, but it cannot rescue a router that has already committed to the wrong experts and is sure of its choice. A dense token costs E/k times a sparse one.

4.7 Threshold selection instead of top- k (threshold)

Instead of always taking exactly k experts, this strategy takes every expert whose router probability is above a threshold, and always keeps the single most likely expert so that no token is left with none:

$$\mathcal{S}(x) = \{i : p_i > \tau/E\} \cup \{\arg \max_i p_i\}, \quad \tau(s) = \tau_0 + \min\left(1, \frac{s}{0.8S}\right)(\tau_1 - \tau_0). \quad (16)$$

The threshold is set relative to an even split. $1/E$ is the probability each expert would get if the router spread its probability evenly, and the threshold is τ times that. τ rises from $\tau_0 = 1$ to $\tau_1 = 1.6$ over the first 80% of training, so with $E = 8$ the threshold rises from 0.125 to 0.2. A confident token therefore keeps one expert, while an unsure one keeps several. Renormalising over a single selected expert gives the constant gate 1 and a zero router gradient, so the gate falls back per token to $g_i = p_i$ whenever $|\mathcal{S}(x)| = 1$. The threshold range was calibrated from the trained router’s measured probabilities (top-1, 2, 3 masses of roughly 0.39, 0.17, 0.12 at $E = 8$); an absolute threshold near 0.5 cannot work because at most one expert can exceed it. This is a sharpening rule, and the only strategy whose mechanism could plausibly *worsen* collapse: an expert that falls behind clears the bar for fewer tokens. It is also the only one whose routing rule stays in force at evaluation, so its inference cost varies per token. It is closely related to top- p routing [13] and to ReLU routing [40].

4.8 Two configuration choices that decide whether collapse is possible

Gate normalisation. The renormalised gate expresses only relative preference among the chosen experts and removes much of the rich-get-richer pressure by construction. At $k = 1$ it is a softmax over a single number, the constant 1, so the router receives *no task gradient* and never trains; a frozen router looks perfectly balanced. We use the raw gate at $k = 1$.

Capacity factor. With no capacity limit, collapse wastes parameters but does not cost accuracy, since every token still reaches its top- k . A capacity factor couples the two: a collapsed router overflows its favourite experts and drops real tokens. It is also itself an anti-collapse force (section 6.4).

5 Experimental setup

Model and data. A GPT-style decoder [32] with 4 layers, $d = 128$, 4 heads, context 128, and an MoE feed-forward block in every layer with $E = 8$ experts of hidden width $d_h = 2d = 256$ and GELU activations. The model has 2.40M parameters, 2.11M of them in experts, and 0.82M active per token at $k = 2$. We train on tiny-shakespeare [17] at the character level (vocabulary 65; 1.00M training and 0.11M validation characters) with AdamW [22] (learning rate 3×10^{-4} , weight decay 0.1, 5% linear warm-up, cosine decay to 10%).

Fairness rules. Every strategy shares architecture, initialisation seed, data order and a frozen evaluation set. The *token* budget is held constant by default, and the learning-rate schedule is indexed by token progress so that accumulation does not change its shape; section 6.3 reverses this and holds optimiser steps constant. All strategies except **threshold** are evaluated with greedy top- k and exploration off, as they would be deployed.

Metrics. Quality is validation cross-entropy and the share of assignments dropped for capacity [8]. Collapse is H_{norm} , max load and dead experts, pooled and per layer, from the held-out dispatch histogram. Compute is analytic, following the accounting of Kaplan et al. [16] and Hoffmann et al. [12]: FLOPs are 2× MACs, backward is charged at 2× forward, and expert cost is driven by the *measured* number of (token, expert) evaluations, so dense or variable- k routing is priced exactly.

Regimes. **Exp. 1:** top-2, renormalised gate, $\gamma = 1.25$, 3,000 steps of batch 32 (12.3M tokens per strategy). **Exp. 2:** top-1, raw gate, $\gamma = 1.25$, learning rate 10^{-3} , batch 16, 3,000 steps. This is the Switch-style regime, in which collapse has room to run. **explore** is excluded as it requires $k \geq 2$. **Exp. 3:** as Exp. 1 but with 1,000 optimiser steps for every strategy, so **accum** consumes 8× the tokens. All runs use seed 0 on a single T4 GPU (about one hour in total).

6 Results

6.1 Exp. 1: top-2, renormalised gate

Table 1 shows the results of Exp. 1. The auxiliary loss gives the flattest router ($H_{\text{norm}} = 0.999$) and essentially eliminates dropping (0.01% against 12.55% for the control) at the FLOP cost of an ideal top- k run; the control is cheaper only because it silently refused an eighth of its expert work. **conf** has the best loss but the second-worst router: its layer 0 ends at $H_{\text{norm}} = 0.772$ with three dead experts, and it drops *more* than the control. Its quality comes from the dense phase training every expert, not from better routing (fig. 1). **explore** with $\epsilon_0 = 0.6$ costs quality without improving H_{norm} , although it does lower the load on the busiest experts (max load falls from 0.172 to 0.152 and dropping from 12.6% to 1.8%). **surrogate** helps here: dropping falls from 12.6% to 3.0% and H_{norm} rises to 0.994, at 0.95× the cost. **threshold** routes only 1.11 experts per token at evaluation, 44% fewer than top-2, for about 1% higher loss than the control.

Table 1: Exp. 1 (top-2, renormalised gate, $\gamma = 1.25$, equal tokens). “vs top- k ” is training FLOPs relative to an idealised top- k run with no dropping. Best per column in bold. Single seed: quality differences of a few percent of perplexity are within noise.

strategy	val loss ↓	H_{norm} ↑	max load ↓	dropped ↓	TFLOPs	vs top- k	experts/tok
conf	1.658	0.973	0.195	13.46%	90.3	1.42×	1.73
aux	1.663	0.999	0.138	0.01%	63.6	1.00×	2.00
control	1.698	0.986	0.172	12.55%	57.7	0.91×	1.75
surrogate	1.711	0.994	0.165	3.03%	60.8	0.95×	1.94
threshold	1.719	0.959	0.216	19.02%	52.2	0.82×	1.11
explore	1.766	0.987	0.152	1.82%	62.5	0.98×	1.96
accum	2.327	0.958	0.213	19.79%	53.5	0.84×	1.60

6.2 Exp. 2: top-1, raw gate

Table 2 shows the results of Exp. 2. The auxiliary loss reaches $H_{\text{norm}} = 1.000$ and cuts dropping from 15.8% to 0.37%. **surrogate** fails badly: loss 1.888, 28.5% dropped and eleven dead experts across four layers, with layer 3 at $H_{\text{norm}} = 0.640$ (fig. 3). Under the raw gate the unselected logits already receive gradient through the full-softmax normaliser, so the surrogate adds a second, redundant pressure that is collinear with the evaluated output and pushes every unselected expert in the same direction regardless of merit. Helpful as the only signal (Exp. 1), harmful as a redundant one.

6.3 Exp. 3: equal optimiser steps

Table 3 shows the results of Exp. 3. Given equal updates, accumulation’s hypothesis holds: loss 1.884 against 2.166, H_{norm} 0.989 against 0.962, dropping 6.5% against 17.0%. It is, however, the least compute-efficient: 156.3 TFLOPs to reach 1.884, whereas **aux** in Exp. 1 reaches 1.663 on 63.6 TFLOPs. Its advantage is per update, which matters only when updates rather than FLOPs are scarce.

Table 2: Exp. 2 (top-1, raw gate, $\gamma = 1.25$). “Dead” counts experts below 10% of a fair share, summed over the four layers; the pooled count is zero for every strategy.

strategy	val loss ↓	$H_{\text{norm}} \uparrow$	max load ↓	dropped ↓	TFLOPs	vs top- k	dead (Σ layers)
aux	1.609	1.000	0.134	0.37%	21.9	0.99×	0
conf	1.610	0.973	0.213	13.75%	43.4	1.95×	0
threshold	1.619	0.976	0.200	19.63%	23.7	1.07×	1
control	1.635	0.969	0.202	15.79%	19.0	0.85×	0
surrogate	1.888	0.965	0.194	28.51%	19.2	0.86×	11
accum	2.220	0.962	0.206	27.88%	18.5	0.83×	4

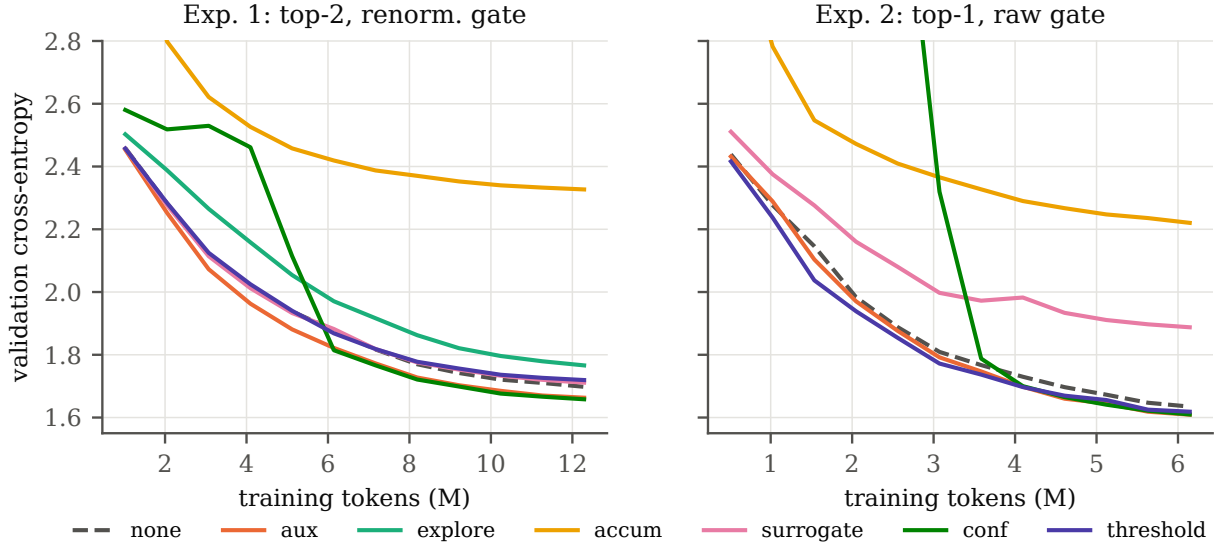


Figure 1: Validation loss against training tokens. *conf* stays near 2.5 while it routes densely and evaluation (always top- k) does not match training, then drops sharply as its threshold anneals. *accum* is undertrained at equal tokens, taking one-eighth of the optimiser steps.

Table 3: Exp. 3 (as Exp. 1, but 1,000 optimiser steps for every strategy).

strategy	val loss ↓	$H_{\text{norm}} \uparrow$	max load ↓	dropped ↓	TFLOPs	tokens
accum	1.884	0.989	0.167	6.49%	156.3	32.8M
aux	2.105	0.998	0.137	0.17%	21.1	4.1M
conf	2.128	0.991	0.167	6.64%	33.0	4.1M
control	2.166	0.962	0.207	17.01%	18.4	4.1M
threshold	2.173	0.992	0.177	11.52%	18.1	4.1M
surrogate	2.178	0.981	0.184	20.45%	18.0	4.1M
explore	2.183	0.988	0.178	5.18%	20.5	4.1M

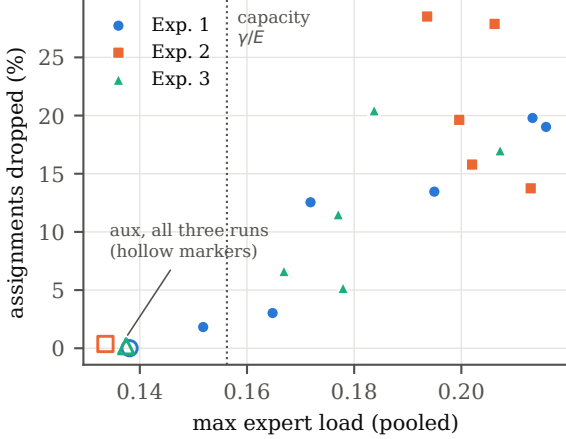


Figure 2: Pooled max load against dropped assignments for every run; hollow markers are aux. Dropping rises with the busiest expert’s share relative to the capacity line $\gamma/E = 0.156$, but pooled max load is only a loose predictor, because capacity is enforced per layer and per batch.

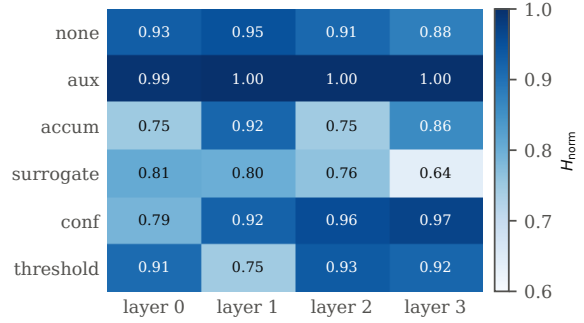


Figure 3: Per-layer H_{norm} in Exp. 2. Pooled H_{norm} for surrogate is a healthy-looking 0.965 while its last layer has collapsed to 0.640.

6.4 Cross-cutting findings

Max load predicts dropping; entropy saturates. Across all runs H_{norm} lies between 0.958 and 1.000 and barely discriminates, while dropping tracks the busiest expert’s share against the capacity line almost exactly in Exp. 1 (fig. 2). Max load is the operationally meaningful collapse metric.

Pooling hides collapse. Every run reports zero dead experts when pooled over layers; per layer, the same runs lose up to eleven. Model-level averages are not collapse diagnostics.

A capacity factor is itself an anti-collapse force. Re-running Exp. 3 without a capacity limit, accum’s advantage in H_{norm} over the control vanished (0.973 vs 0.970, against 0.989 vs 0.962 with the limit) and threshold’s entropy declined monotonically through training (0.963→0.930), the only strategy to do so. Dropping lowest-gate-first denies an overloaded expert its marginal tokens and therefore its marginal gradient.

None of the metrics measure specialisation. Every quantity above is a function of the load histogram. As section 3.3 explains, the aux router’s $H_{\text{norm}} = 0.999$ is consistent with a well-specialised partition and with near-random routing alike. The experiments show which strategies prevent collapse; they say nothing about which produce experts.

7 Discussion

At this scale the auxiliary loss is the strongest single method on every axis we measured, and it is untuned. The alternatives that use no auxiliary loss each contribute a mechanism that works somewhere: conf matches or beats the auxiliary loss on quality without any balancing loss, paying extra only during training, although we cannot yet say whether its per-token confidence rule adds anything over a plain dense warm-up; surrogate shows that dense router gradients help when they are the router’s only signal; accum shows that lower-variance router updates reduce collapse; explore lowers the load on the busiest experts, which sharply cuts dropped assignments; and threshold shows that the number of experts needed per token is often much less than k . Each is also worse than doing nothing in at least one setting: on quality for all except conf, which beats the control on validation loss everywhere but leaves the router less balanced in Exp. 1.

We read these results as consistent with the paper’s premise rather than as evidence against it. The auxiliary loss is best at the one thing it optimises, an even spread of load, and that is all our metrics measure. Whether any of

these methods produces the specialisation that hypothesis 1 requires is exactly the question the pilot could not answer. The literature is suggestive but not conclusive. Hash and random routing are competitive with learned routing [7, 34, 43], Mixtral shows weak domain specialisation [15], and expert activations in deeper layers are near-identical across semantically unrelated inputs, especially in reasoning models [39]. Together these suggest that current learned routers are closer to balanced hashes than to partitions. Relaxing the *scope* of balancing from micro-batch to global batch produces clearly more specialised experts and better models [31], and decoupling balancing from the gradient improves quality [38], which suggests that the balancing term actively suppresses specialisation that the task gradient would otherwise produce.

The random-routing results also admit a reading that cuts against our hypothesis. Random routing may match learned routing because experts at current sizes are large enough to handle any mix of tokens, in which case how tokens are partitioned does not matter much and better routing would not let experts shrink. Zuo et al. [43] also observe near-random gating both with and without a load-balancing loss, so the balancing loss is unlikely to be the only reason routers fail to specialise. None of these studies varies expert size, so none can separate the two readings. The shrinkage experiment (section 8.4) is designed to do exactly that: our hypothesis predicts that specialised routing pulls ahead of random routing as experts shrink, while the alternative predicts that it does not.

7.1 Limitations

All results come from one seed, one 2.4M-parameter model, one small homogeneous character-level corpus, and at most 3,000 optimiser steps per run. Quality differences of a few percent of perplexity are not significant. Hyper-parameters of every strategy, including α , are untuned. We measure balance, not specialisation, and the shrinkage hypothesis is argued, not demonstrated. Compute is reported analytically and excludes dispatch overheads that matter in practice. Our argument in section 3.3 concerns the routers that minimise the loss; it does not claim that auxiliary-loss training *converges* to random routing, only that the loss provides no pressure away from it.

8 A protocol for testing specialisation and shrinkage

We describe the experiment we would run with adequate compute. Its purpose is to test hypothesis 1 directly and to rank strategies on specialisation, not only on balance.

8.1 Scale and data

- **Tokenisation and corpus.** Subword tokens on a large, *domain-labelled* mixture (e.g. web text, code, mathematics, several natural languages), so that specialisation has something to latch onto and can be measured against ground-truth domains. Character-level Shakespeare is too homogeneous for domain specialisation to exist.
- **Two model scales.** A sweep scale of roughly 100–150M active parameters trained near the compute-optimal token count [12] (~2–3B tokens), and a confirmation scale of about 1B active parameters for the best three configurations. Collapse and specialisation are both known to change with scale and training length, and our 3,000-step runs are far from either regime.
- **More and finer experts.** $E \in \{8, 32, 64\}$ with $k \in \{1, 2, 8\}$, including a fine-grained configuration with shared experts [3].
- **Seeds.** At least three per configuration, reporting mean and standard deviation; single-seed rankings in our tables should not be trusted.

8.2 Baselines

In addition to our seven configurations: global-batch auxiliary loss [31]; auxiliary-loss-free bias balancing [38]; Default MoE dense backpropagation [30]; expert-choice routing [42]; a frozen random-hash router [34], which is the *null model for specialisation*: any method whose specialisation metrics do not beat it has not learned to route; and a dense model of equal active parameters.

8.3 Specialisation metrics

All computed per layer on held-out data:

1. **Token-expert mutual information** $I(X; \mathcal{E})$ from (6), split into its balance and per-token-entropy terms, normalised by $\ln E$.
2. **Domain-expert mutual information** $I(D; \mathcal{E})$ [9] using the corpus’s domain labels, together with per-domain dispatch histograms in the style of Muennighoff et al. [26].
3. **Routing stability**: the fraction of held-out tokens routed to the same top-1 expert at checkpoint s and at the final checkpoint, which separates a stable partition from high-entropy churn [2].
4. **Functional specialisation**: the increase in loss when expert i is removed [5], measured separately on each domain. Dai et al. [3] use a related test, disabling the top routed experts to measure redundancy. A specialised expert has a large, domain-concentrated ablation effect; a redundant one has a small, uniform effect.
5. **Expert redundancy**: mean pairwise linear CKA [19] between experts’ outputs on a common probe set of inputs. Redundant generalists have high CKA; hypothesis 1 predicts that CKA falls as $I(X; \mathcal{E})$ rises.
6. **Collapse metrics** as in this paper, always per layer, with max load reported alongside H_{norm} .

8.4 The shrinkage experiment

The decisive test of hypothesis 1. For each routing procedure $\mathcal{R}$, train models whose expert width is d_h/r for $r \in \{1, 2, 4, 8\}$, holding E, k , the backbone and the token budget fixed, and plot validation loss against total expert parameters $E d_h/r$. The hypothesis predicts that the loss-versus-memory curve is flatter for procedures with higher measured specialisation. Two complementary controls: (i) at fixed *active* compute, reduce E instead of d_h , to separate “fewer experts” from “smaller experts”; and (ii) apply post-hoc merging [21] and pruning [23] to the $r = 1$ models, measuring how much each procedure’s experts can be compressed after training. If specialised routing yields experts that compress *less* post hoc but can be trained *smaller* a priori, that is strong evidence for the hypothesis.

8.5 Ablations that our pilot flagged

- Tune α for the auxiliary loss, including very small values: its dominance here is untuned.
- explore with a lower starting rate $\epsilon_0 \in \{0.1, 0.2, 0.3\}$, since 0.6 disrupts more than it helps, and a floor ϵ_1 higher than the current 0.01.
- A plain dense warm-up of 10–20% of training, to isolate whether conf’s gains come from confidence gating or simply from dense early training.
- surrogate only with the renormalised gate, compared with Default MoE’s per-expert estimate.
- With and without capacity factor, since the limit itself suppresses collapse.

8.6 Compute estimate

Using the $6ND$ approximation [16] with N active parameters, a 125M-active model trained on 2.5B tokens costs about 1.9×10^{18} FLOPs; at an effective 10^{14} FLOP/s on one A100 (roughly 30% utilisation, discounted further for MoE dispatch overhead), that is about 5–10 GPU-hours per run. The sweep (ten routing procedures, four expert widths, three seeds) is 120 runs, or on the order of 1,000 GPU-hours. The confirmation scale ($\sim 1\text{B}$ active parameters, 20B tokens, $\sim 1.2 \times 10^{20}$ FLOPs per run) for three configurations and two seeds adds on the order of 2,000 GPU-hours. The full protocol is therefore in the low thousands of A100-hours: modest for an industrial lab, well beyond the budget used for the pilot.

8.7 Toward a combined method

No single strategy we tested produces routers that are both balanced and specialised, and we do not expect one to. A method that gets experts to specialise properly will most likely need to combine several of the strategies covered here,

and possibly others we have not tested, such as an explicit reward for decisive routing. Designing that combination is the natural next step once the protocol above can measure specialisation.

9 Conclusion

Preventing router collapse and producing specialised experts are different objectives. The standard auxiliary loss is an effective answer to the first, and in our pilot the most effective one, but it is provably indifferent to the second. Specialisation is what would let each expert be smaller and would attack the memory cost that defines MoE deployment. We have described six mechanisms for preventing collapse, characterised where each helps and fails, argued that getting experts to specialise will likely require combining several of them, and set out the experiment that would test whether specialised routers really allow smaller experts. The code for every experiment in this paper is available at <https://github.com/muadmazahir/MoE-experiments>, so that the protocol can be run by anyone with the compute to do so.

References

- [1] Yoshua Bengio, Nicholas Léonard, and Aaron Courville. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432*, 2013.
- [2] Damai Dai, Li Dong, Shuming Ma, Bo Zheng, Zhifang Sui, Baobao Chang, and Furu Wei. StableMoE: Stable routing strategy for mixture of experts. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2022.
- [3] Damai Dai, Chengqi Deng, Chenggang Zhao, R. X. Xu, Huazuo Gao, Deli Chen, Jiashi Li, Wangding Zeng, Xingkai Yu, Y. Wu, et al. DeepSeekMoE: Towards ultimate expert specialization in mixture-of-experts language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1280–1297, 2024.
- [4] DeepSeek-AI, Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, et al. DeepSeek-V3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.
- [5] Leonard Engmann, Christian Medeiros Adriano, and Holger Giese. From observation to intervention: A causal audit of expert importance in mixture-of-experts models. In *ICML 2026 Workshop on Philosophy of Science Meets Machine Learning (PhilML)*, 2026. Non-archival.
- [6] Tobias Falke, Nicolas Anastassacos, Samson Tan, Chankrisna Richy Meas, Chandana Satya Prakash, Nitesh Sekhar, et al. MoE routing testbed: Studying expert specialization and routing behavior at small scale. *arXiv preprint arXiv:2604.07030*, 2026.
- [7] Dongyang Fan, Bettina Messmer, and Martin Jaggi. Towards an empirical understanding of MoE design choices. *arXiv preprint arXiv:2402.13089*, 2024.
- [8] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.
- [9] Naibin Gu, Zhenyu Zhang, Yuchen Feng, Yilong Chen, Peng Fu, Zheng Lin, Shuohuan Wang, Yu Sun, Hua Wu, Weiping Wang, and Haifeng Wang. Elastic MoE: Unlocking the inference-time scalability of mixture-of-experts. *arXiv preprint arXiv:2509.21892*, 2025.
- [10] Hongcan Guo, Haolang Lu, Guoshun Nan, Bolun Chu, Jialin Zhuang, Yuan Yang, et al. Advancing expert specialization for better MoE. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.
- [11] Xu Owen He. Mixture of a million experts. *arXiv preprint arXiv:2407.04153*, 2024.
- [12] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, et al. Training compute-optimal large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

- [13] Quzhe Huang, Zhenwei An, Nan Zhuang, Mingxu Tao, Chen Zhang, Yang Jin, Kun Xu, Liwei Chen, Songfang Huang, and Yansong Feng. Harder tasks need more experts: Dynamic routing in MoE models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.
- [14] Robert A. Jacobs, Michael I. Jordan, Steven J. Nowlan, and Geoffrey E. Hinton. Adaptive mixtures of local experts. *Neural Computation*, 3(1):79–87, 1991.
- [15] Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, et al. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024.
- [16] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.
- [17] Andrej Karpathy. char-rnn: Multi-layer recurrent neural networks for character-level language models. <https://github.com/karpathy/char-rnn>, 2015. Source of the tiny-shakespeare corpus.
- [18] Kimi Team, Yifan Bai, Yiping Bao, Y. Charles, Cheng Chen, Guanduo Chen, et al. Kimi K2: Open agentic intelligence. *arXiv preprint arXiv:2507.20534*, 2025.
- [19] Simon Kornblith, Mohammad Norouzi, Honglak Lee, and Geoffrey Hinton. Similarity of neural network representations revisited. In *International Conference on Machine Learning (ICML)*, 2019.
- [20] Dmitry Lepikhin, HyukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. GShard: Scaling giant models with conditional computation and automatic sharding. In *International Conference on Learning Representations (ICLR)*, 2021.
- [21] Pingzhi Li, Zhenyu Zhang, Prateek Yadav, Yi-Lin Sung, Yu Cheng, Mohit Bansal, and Tianlong Chen. Merge, then compress: Demystify efficient SMoE with hints from its routing policy. In *International Conference on Learning Representations (ICLR)*, 2024.
- [22] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.
- [23] Xudong Lu, Qi Liu, Yuhui Xu, Aojun Zhou, Siyuan Huang, Bo Zhang, Junchi Yan, and Hongsheng Li. Not all experts are equal: Efficient expert pruning and skipping for mixture-of-experts large language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.
- [24] Jan Ludziejewski, Jakub Krajewski, Kamil Adamczewski, Maciej Pióro, Michał Krutul, Szymon Antoniak, Kamil Ciebiera, Krystian Król, Tomasz Odrzygóźdź, Piotr Sankowski, Marek Cygan, and Sebastian Jaszczur. Scaling laws for fine-grained mixture of experts. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *Proceedings of Machine Learning Research*, pages 33270–33288, 2024.
- [25] Jama Hussein Mohamud, Drew Wagner, and Mirco Ravanelli. Self-routing: Parameter-free expert routing from hidden states. *arXiv preprint arXiv:2604.00421*, 2026.
- [26] Niklas Muennighoff, Luca Soldaini, Dirk Groeneveld, Kyle Lo, Jacob Morrison, Sewon Min, et al. OLMoE: Open mixture-of-experts language models. In *International Conference on Learning Representations (ICLR)*, 2025.
- [27] Basil Mustafa, Carlos Riquelme, Joan Puigcerver, Rodolphe Jenatton, and Neil Houlsby. Multimodal contrastive learning with LIMoE: the language-image mixture of experts. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 9564–9576, 2022.
- [28] Xiaonan Nie, Xupeng Miao, Shijie Cao, Lingxiao Ma, Qibin Liu, Jilong Xue, et al. EvoMoE: An evolutionary mixture-of-experts training framework via dense-to-sparse gate. *arXiv preprint arXiv:2112.14397*, 2021.
- [29] Nabil Omi, Siddhartha Sen, and Ali Farhadi. Load balancing mixture of experts with similarity preserving routers. *arXiv preprint arXiv:2506.14038*, 2025.

- [30] Ashwinee Panda, Vatsal Baherwani, Zain Sarwar, Benjamin Therien, Sambit Sahu, Tom Goldstein, and Supriyo Chakraborty. Dense backpropagation improves training for sparse mixture-of-experts. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.
- [31] Zihan Qiu, Zeyu Huang, Bo Zheng, Kaiyue Wen, Zekun Wang, Rui Men, Ivan Titov, Dayiheng Liu, Jingren Zhou, and Junyang Lin. Demons in the detail: On implementing load balancing loss for training specialized mixture-of-expert models. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025.
- [32] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. Technical report, OpenAI, 2019.
- [33] Carlos Riquelme, Joan Puigcerver, Basil Mustafa, Maxim Neumann, Rodolphe Jenatton, André Susano Pinto, Daniel Keysers, and Neil Houlsby. Scaling vision with sparse mixture of experts. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 34, pages 8583–8595, 2021.
- [34] Stephen Roller, Sainbayar Sukhbaatar, Arthur Szlam, and Jason Weston. Hash layers for large sparse models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.
- [35] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations (ICLR)*, 2017.
- [36] Yikang Shen, Zheyu Zhang, Tianyou Cao, Shawn Tan, Zhenfang Chen, and Chuang Gan. ModuleFormer: Modularity emerges from mixture-of-experts. *arXiv preprint arXiv:2306.04640*, 2023.
- [37] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018.
- [38] Lean Wang, Huazuo Gao, Chenggang Zhao, Xu Sun, and Damai Dai. Auxiliary-loss-free load balancing strategy for mixture-of-experts. *arXiv preprint arXiv:2408.15664*, 2024.
- [39] Xi Wang, Soufiane Hayou, and Eric Nalisnick. The myth of expert specialization in MoEs: Why routing reflects geometry, not necessarily domain expertise. *arXiv preprint arXiv:2604.09780*, 2026.
- [40] Ziteng Wang, Jun Zhu, and Jianfei Chen. ReMoE: Fully differentiable mixture-of-experts with ReLU routing. In *International Conference on Learning Representations (ICLR)*, 2025.
- [41] Heng Zhao, Zilei Shao, Guy Van den Broeck, and Zhe Zeng. ProbMoE: Differentiable probabilistic routing for mixture-of-experts. In *International Conference on Machine Learning (ICML)*, 2026.
- [42] Yanqi Zhou, Tao Lei, Hanxiao Liu, Nan Du, Yanping Huang, Vincent Zhao, Andrew Dai, Zhifeng Chen, Quoc Le, and James Laudon. Mixture-of-experts with expert choice routing. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [43] Simiao Zuo, Xiaodong Liu, Jian Jiao, Young Jin Kim, Hany Hassan, Ruofei Zhang, et al. Taming sparsely activated transformer with stochastic experts. In *International Conference on Learning Representations (ICLR)*, 2022.

A Hyper-parameters

Table 4 lists the settings shared by every run and table 5 the defaults for each strategy. S is the total number of training steps.

Table 4: Model and training settings shared by every run. Where Exp. 2 differs, its value is given in brackets.

setting	value
layers, width d , heads, context	4, 128, 4, 128
experts E , experts per token k	8, 2 (1)
expert hidden width d_h	256
gate	renormalised (raw)
dropout	0
optimiser	AdamW, $\beta = (0.9, 0.95)$
learning rate	3×10^{-4} (10^{-3})
schedule	5% linear warm-up, cosine decay to 10%
weight decay	0.1, on weight matrices only
gradient clipping	1.0 (global norm)
batch size	32 (16) sequences of 128 characters
training steps	3,000 (Exp. 3: 1,000 optimiser steps)
evaluation	every 250 steps, on 20 fixed held-out batches
seed	0

Table 5: Default strategy hyper-parameters.

strategy	hyper-parameter	value
aux	loss weight α	10^{-2}
explore	$\epsilon_0 \rightarrow \epsilon_1$, anneal fraction ρ	$0.6 \rightarrow 0.01, 0.6S$
	gate temperature T (exploring tokens only)	4 (constant)
accum	micro-batches per step M	8
surrogate	straight-through weight α_s	1.0
conf	τ_c^0 , annealed to 0 over	0.8, $0.8S$
threshold	$\tau_0 \rightarrow \tau_1$ (units of $1/E$), rise over	$1.0 \rightarrow 1.6, 0.8S$
all	capacity factor γ	1.25

B Reproducing the experiments

The code is available at <https://github.com/muadmazahir/MoE-experiments>. From the repository root, the three experiments and the figures are reproduced with:

```
# Exp. 1: top-2, renormalised gate, equal tokens
python run_experiment.py --steps 3000 --capacity-factor 1.25 \
  --seeds 0 --out results/exp1-seed0
# Exp. 2: top-1, raw gate
python run_experiment.py --k 1 --gate-norm raw --capacity-factor 1.25 \
  --lr 1e-3 --batch-size 16 --steps 3000 \
  --methods none,aux,accum,surrogate,conf,threshold \
  --seeds 0 --out results/exp2-seed0
# Exp. 3: equal optimiser steps
python run_experiment.py --equalize steps --steps 1000 --capacity-factor 1.25 \
  --seeds 0 --out results/exp3-seed0
# Figures
python paper/figures/make_figures.py
```